A Bottom Up Approach for Modeling Business Process using Time Petri Nets

Danillo S. Ramos
danillo.ramos@dcomp.ufs.br
Universidade Federal de Sergipe
São Cristóvão, Sergipe, Brasil

Fabio G. Rocha
Universidade Tiradentes
Aracaju, Sergipe, Brasil
gomesrocha@gmail.com

Michel S. Soares
michel@dcomp.ufs.br
Universidade Federal de Sergipe
São Cristóvão, Sergipe, Brasil

ABSTRACT

Context: UML is an important modeling language for software development in industry, considered a *de facto* standard for specifying a variety of models, including business processes, data models, and user scenarios that represent software at a high level of abstraction. **Problem:** Although UML provides the user with several diagrams for software modeling, the language still leaves gaps, such as a poor capacity for modeling time constraints, and simulating the desired behaviour of software. Thus, using only UML to model distributed systems developed using microservices, and also modeling time constraints and system's behaviour becomes an issue that has been identified by many researchers. **Proposed solution:** Seeking to mitigate UML gaps, an Electronic Document Management System (SGED) modeled using UML was transcribed to Petri nets with time, using a bottom-up approach. In addition, formal verification and analysis of the model's properties were performed. **Results:** Through the Petri Nets model with time, it was possible to analyze the behaviour of the modelled system, as well as analyze the systems' properties, and then detect and prevent possible unwanted behaviour. **Contributions:** As for conclusion, it is clear the importance of using formal methods in practice, specially Petri nets, in systems with microservices where time constraints are crucial for their correct behaviour. In addition, it is feasible to construct large models by composing Petri nets, using a bottom-up approach, which corresponds nicely with the architecture of microservices, with independent, loosely coupling services.

CCS CONCEPTS

• **Computer systems organization** → **Information systems**; *Software Architecture*; Microservices; • **Networks** → Network reliability.

KEYWORDS

Electronic Documents, Time Petri Nets, UML, Business Process, Microservices

ACM Reference Format:
Danillo S. Ramos, Fabio G. Rocha, and Michel S. Soares. 2022. A Bottom Up Approach for Modeling Business Process using Time Petri Nets. In *XVIII Brazilian Symposium on Information Systems (SBSI), May 16–19, 2022, Curitiba, Brazil.* ACM, New York, NY, USA, Article 111, 8 pages. https://doi.org/10.1145/3535511.3535539

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SBSI, May 16–19, 2022, Curitiba, Brazil
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9698-1/22/05…$15.00
https://doi.org/10.1145/3535511.3535539

1 INTRODUCTION

A Business Process is a set of activities with one or more types of input, creating an output that is of value to the customer [Wang et al. 2022]. Business processes are known to be of fundamental importance in reaping the benefits brought to any organization that is using Information Systems. Business Processes have been adopted in distributed systems based on the microservices architecture, aiming at more agility and ease of maintenance. Such an architectural style consists of decomposing software applications into small independent and asynchronous services, which communicate by exchanging messages [Balalaie et al. 2018; Mazlami et al. 2017; Valderas et al. 2022]. Microservices have been employed primarily when related to collaboration to meet business process models [Alpers et al. 2015].

Methods for process improvement significantly impact the organization's existing processes or the development of new processes [Malinova et al. 2022]. Digitalization of services within the organization can be cited as an improvement of business processes, being adopted by many governmental and non-governmental organizations [Almarabeh et al. 2016; Mohamad et al. 2022]. In government organizations, the use of Information and Communication Technologies (ICT) has promoted a significant evolution of the services offered to society, providing greater transparency, cost reduction, and increase in quality and speed of service [Kurfalı et al. 2017]. However, despite the benefits offered by the implementation of ICT services, organizations are facing some difficulties, such as the proper administration of ICT resources, whether material or human [Sohal and Fitzpatrick 2002], as well as the rapid pace of technology change.

Traditionally, ICT in government organizations is directed towards the development and application of internal services, such as management of electronic documents [Das et al. 2017; Janowski 2015]. The purpose of electronic document management is to facilitate the flow of records and ensure the greater availability of requested information. It also helps the organization to achieve more efficient operations, reduce transaction costs, automate processes, minimize errors and save labor. Despite all the advantages offered, an electronic document management system requires high costs and efforts for development, management and deployment [Alshibly et al. 2016].

1.1 Related Works

Business processes have been modelled using a variety of modeling languages and diagrams. For instance, UML Activity diagram [Cerbo et al. 2011], BPMN [Arévalo et al. 2016], Flowcharts [Schäfer et al. 2021] and IDEF0 [Baghbani 2019]. Despite being considered a standard modeling language, UML lacks some important features for various types of systems, such as difficulties for modeling time constraints and formal verification [Brant-Ribeiro et al. 2019; Fernández-Sáez et al. 2013]. Furthermore, the same system can be described by multiple UML diagrams, leading to an inconsistent or ambiguous model [Alhumaidan and Zafar 2014].

All these notations and diagrams are capable of modeling business processes, but they lack formal semantics for properties verification such as the absence of livelocks or deadlocks. Therefore, one option is to use a formal modeling language that addresses this issue.

One common option is to model the business processes using one of the many Petri Nets types [Ramadan et al. 2011; Rogge-Solti and Weske 2015; Zimmermann 2012]. For instance, the authors of paper [Mutarraf et al. 2018] proposed an approach to transform a Business Process Model and Notation diagram automatically to Petri nets, then add time to transitions and probability to gateways, and furthermore simulating time using MATLAB. The variety of Petri net-based extensions for business process modeling shows that Petri nets can be adapted to changing requirements for which these extensions, modifications or variants have been proposed [Koschmider et al. 2018].

Despite its many advantages, formal methods have also their drawbacks and well-known challenges. For instance, shortage of qualified personnel [Polak 2002] and general difficulties with tools, including issues of scalability and integration [Davis et al. 2013]. Therefore, one suitable solution is to combine formal models with other software/systems notations [Soares and Vrancken 2008], as presented in this paper, in which UML Activity diagrams are first used for business process modeling, followed by equivalent Petri net models including timing constraints.

1.2 Research Approach

In this paper, we present a UML Activity diagram model for process behaviour of an electronic document management system - named SGED - , as well as the transcription for a Petri net model with time. The system presented is used to manage medical certificates, check deadlines and validate documents.

This work is characterized as exploratory research, as a case study is carried out. According to Swedberg [Swedberg 2020], exploratory research can be defined as a type of research in which researchers seek proximity to the reality of the object of study, as well as the most appropriate methods and techniques for carrying out the research. A case study, such as the one presented in this paper, is characterized as exploratory research.

In order to evaluate whether the SGED meets the proposed needs, we transcribed the UML model to a Time Petri net model which formalizes the related business process, and gives opportunities for verification of systems properties. The approach presented in this paper follows a bottom-up modeling approach [Soares and Vrancken 2012], which facilitates scaling up as new functionalities are added to the system.

As the main advantage of the adopted approach, we can highlight the construction of small parts of the system, facilitating its composition and preventing some parts from being forgotten [Girault and Valk 2002].

2 THEORETICAL BACKGROUND

This section introduces concepts about UML, Petri Nets with time and microservices architecture.

2.1 Unified Modeling Language (UML)

UML was proposed as a general purpose software modeling language, by combining existing notations and diagrams, so that it provides different points of view of the software to be developed [Fuentes-Fernández and Vallecillo-Moreno 2004]. One advantage of using UML diagrams is that it allows software engineers to visualize the design before starting the actual source code implementation. Current version of UML is 2.5.1, and it classifies diagrams into two groups: structural diagrams, which show the static structure of the software and its parts, at different levels of abstraction; and behavioral diagrams, which show the operation of the software at runtime.

Despite trying to position itself as a general-purpose language for modeling software systems, UML is considered quite complex and not free from ambiguities [Soares and Cioquetta 2012].

2.2 Petri Nets with time

Petri nets are a formal and graphical modeling language, applicable to many systems and domains, including transport, manufacturing, information systems, and all kinds of real-time systems [Abdellatif et al. 2019; Lu et al. 2017; Soares and Vrancken 2008, 2007].

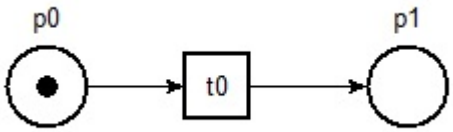


Figure 1: Petri Net example

The structure of a Petri net is composed of places, graphically represented by circles and meaning the actions of the system; transitions, graphically represented by rectangles or boxes, meaning system events; and arcs, graphically represented by unidirectional arrows, connecting places to transitions and vice versa. Places can only be connected to transitions, never to other places. The same goes for transitions, which can only connect to places, never to other transitions. Places can contain marks, called tokens. If the conditions for firing a transition are satisfied, the tokens are moved from one place to another via the arcs [Murata 1989]. Figure 1 shows an example of a Petri Net.

Formally, a Petri net [Murata 1989] is defined as a 5-tuple - (P, T, F, W, M_0), in which:

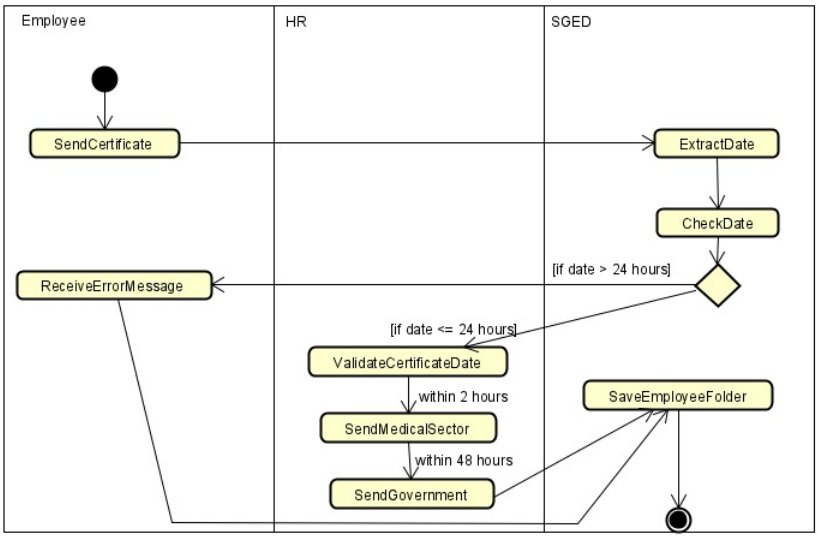


Figure 2: SGED UML Model

- P is a set of finite places,
- T is a set of finite transitions,
- $F \subseteq (P \times T) \cup (T \times P)$ is a set of arcs (flow relation),
- $W : F \rightarrow 1, 2, 3, ..., n$ is a weight function,
- $M_0 : P \rightarrow 1, 2, 3, ..., n$ is the initial marking,
- $P \cap T = \oslash$ and $P \cup T \neq \oslash$.

The dynamic behavior (execution) of a Petri net is described by changing markings (from M_0 to M), which is represented by the firing of an enabled transition. A transition t is enabled if each place P that has an input arc to t contains at least the number of tokens equal to the weight of the arcs connecting P to t ($M(p) \geq Pre(p, t)$ for any $p \in P$).

The flow of tokens represent state changing. The firing of an enabled transition t removes from each input place p_i the number of tokens equal to the weight of the directed arc connecting p_i to t, and deposits in each output place p_o the number of tokens equal to the weight of the directed arc connecting t to p_o. Therefore, the firing of a transition yields a new marking $M'(p) = M(p) - Pre(p, t) + Post(p, t)$ for any $p \in P$. The matrix $C = Post - Pre$ is called the incident matrix of the corresponding Petri net.

Petri Nets depicted in Figure 1 are named ordinary Petri nets. In the 1970's, time restrictions were added to ordinary Petri Nets, as well as other additions and extensions, further enhancing Petri Nets [Jones et al. 1977; Kurkovsky and Loganantharaj 2000; Merlin and Farber 1976; Murata 1989; Ramchandani 1974].

Petri Nets with time have some behavioral properties, among which we can highlight with brief definitions [Murata 1989]:

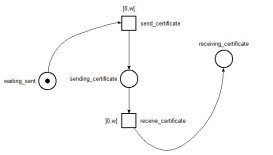
- Reachability: is the basis for studying dynamic properties of any system. A marking M_n is said to be reachable from a marking M_0 if there is a sequence of firings that transform M_0 into M_n, respecting the transition firing rules.
- Boundedness: a Petri Net is said to be k-bounded or simply bounded, if the number of tokens in each place does not exceed a finite number k, for any marking reachable from M_0.
- Liveness: a Petri Net is said to be live if, no matter what marking is reached by M_0, it is possible to fire any transition of the net, progressing through any further firing sequence. Liveness property ensures that the system is deadlock free.
- Reversibility: a Petri Net is said to be reversible if, for each marking M, the initial marking M_0 is reachable from M. This returns the system to its initial state.

2.3 Microservices Architecture

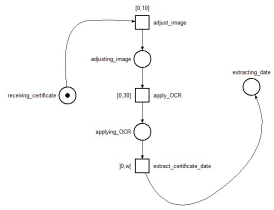
Microservices architecture allows developing software systems as a set of small independent and specific services. Each service communicates with each other through asynchronous message exchange [Balalaie et al. 2018; Mazlami et al. 2017].

The architecture of microservices provides several advantages to the project, such as developing and implementing services independently, providing more effectiveness for the development teams. Services are loosely coupled and have a single task, unlike the monolithic architectures [Jamshidi et al. 2018a; Krylovskiy et al. 2015; Mazlami et al. 2017].

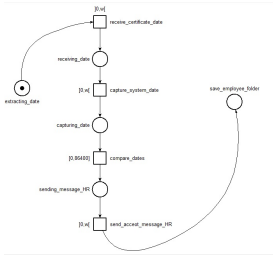
Despite the advantages offered by the microservices architecture, it is also possible to highlight some challenges: there is no way to determine the right size of a microservice, although the name suggests something very small. On this issue, there is a lack of consensus among development teams, as some teams build services with only a few dozen lines of code (LOC) and others encapsulate thousands of LOCs. Front-end integration can also be seen as a challenge, as User Interfaces (UIs) are a critical component in this type of architecture. This is due to the fact that UIs have not been the focus of architecture advocates, who typically deal with the backend aspects, and can lead to systems where a monolithic front end utilizes multiple back end microservices. Occasionally such architectures can work perfectly, but the goals of microservices are



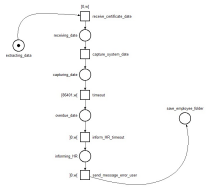
(a) Waiting_Sent Model



(b) Extract_Date Model



(c) Accept_Certificate Model



(d) Timeout Model

Figure 3: Petri Nets Models of SGED

inhibited by the disadvantages of monolithic architecture [Jamshidi et al. 2018b].

3 PROBLEM DESCRIPTION

In order to simulate and verify the behavior of the *Electronic Document Management System* (SGED), we performed the transcription of the UML model of the system to Time Petri nets. Despite the advantages and diversity of diagrams offered by UML, the language does not provide formal mechanisms to verify properties and observe whether the proposed system behaves as expected by stakeholders. Therefore, it is essential to carry out a formal verification, in order to ensure the desired functioning of the modeled system.

3.1 An Example of a UML Activity Diagram for SGED

The model shown in Figure 2, designed using the UML Activity diagram, represents the functioning of part of the system. The first lane represents the employee's activities, the second the activities performed by the HR sector. UML activity diagram was chosen as it best represents activities and tasks in business processes. SGED activities are illustrated in the last lane.

Upon receiving medical certificate sent by an employee, the system first applies Optical Character Recognition (OCR) to extract the date from the document. When extracting this date, the system will check if the certificate is within the required 24-hour period, capturing the system date and time. If the document is within the deadline, the SGED will send confirmation of the validity of the document to HR (Human Resources), which will take the appropriate measures. The SGED will also file the document in the employee's corporate folder. If the document is not within the deadline, the system will send a message informing the employee of the impossibility of receiving that certificate, as well as file it in the employee's folder.

3.2 Petri Net with time model

In order to verify the good properties and check the behavior of the SGED system, the transcription of the UML model to a Petri net model with time is proposed in this research. Figure 3 depicts the models developed in Petri Nets with time.

Figure 3 presents a more detailed model of the UML model shown in Figure 2. In these models (Figure 3), the microservices of the SGED system are specified, and each Petri Net transition represents the action performed by the microservices. The system waits until the user performs the action of sending the certificate. The wait state is represented by the place named "waiting_sent".

In the models in Figure 3, places contains a token, which means that the current state of the system is standby. After the user performs the action of sending the certificate, this token will change place, that is, it will change state. For this to happen, the transition named "send_certificate" must be enabled for firing. The transition will be enabled to be fired if there is a token in a place immediately before it.

3.2.1 Bottom up modeling of microservices. In order to address software complexity, their design must be performed in a modular way, in which parts of the software can be specified and taken into account separately. In software design in general, modularity is a well-known principle, being successfully applied in different domains for many years, realizing the principle of separation of concerns.

Two strategies are most common in the design of modular Petri net models: composition, which is named bottom-up, and refinement, named top-down [Vogler 1992]. Transitions as well as places can represent sub-nets, allowing top-down modeling at several levels of detail [Christensen and Petrucci 2000]. In addition, it is feasible to construct large models by composing Petri nets, using a bottom-up approach [Girault and Valk 2002]. The modularity in this paper is related only to places, being it the refinement or the composition.

A bottom-up modeling strategy usually starts by designing simple models representing parts of the system, and later combining them into more complex ones until the whole model is complete. For instance, places of two diverse Petri nets can be merged (place composition), which creates a single Petri net [Huang and Kirchner

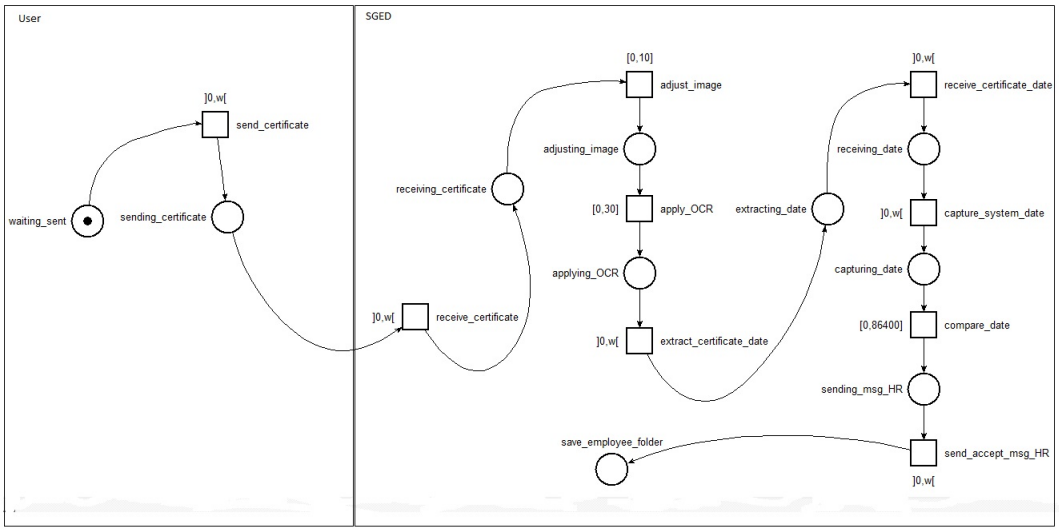


Figure 4: Complete Model in Petri Nets of SGED

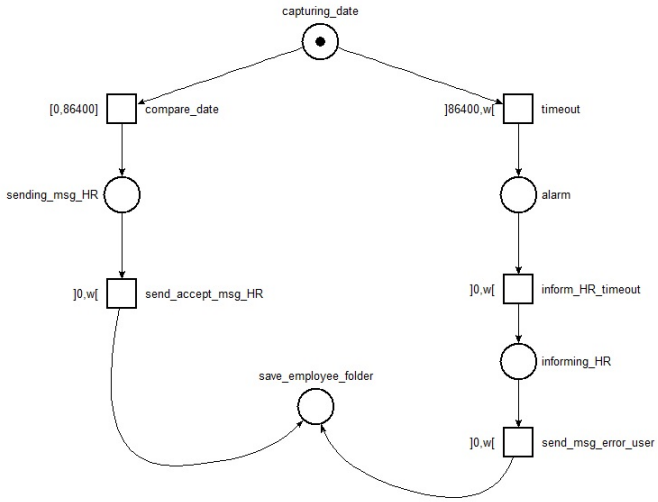


Figure 5: Watchdog

2009]. A classical example is the fusion of common places representing the same resource in two different Petri nets [Cheung et al. 2006], which is a simple and effective way to model communication between subnets [Girault and Valk 2002].

The approach chosen to model the SGED is the bottom-up approach. This approach consists of developing small models of simple parts of the system, which are then combined together to build a larger model that represents the system as a whole. For this, places that represent the same task can be merged into a single place [Jeng and DiCesare 1995]. The smaller SGED models are shown in Figure 3.

The model depicted in Figure 3a represents the standby state of the system. The place called "waiting_sent" has a marking, meaning it will remain in that state until the user requests a document to be sent. While the system is in the waiting state, it also means that the transition "send_certificate" is ready to fire and move the token from the place "waiting_sent" to the place "sending_certificate". It contains a time interval represented by "]0, w[". This time representation means that the transition will fire immediately. In other words, the fire can be done when the user requests the document to be sent to the system. All representations of time intervals in these models are in seconds.

When firing transition "send_certificate", the token will be moved to place "sending_certificate" , which will enable transition "receive_-certificate" (Figure 3b). This last transition also has an immediate fire. When fired, "receive_certificate" will move the token

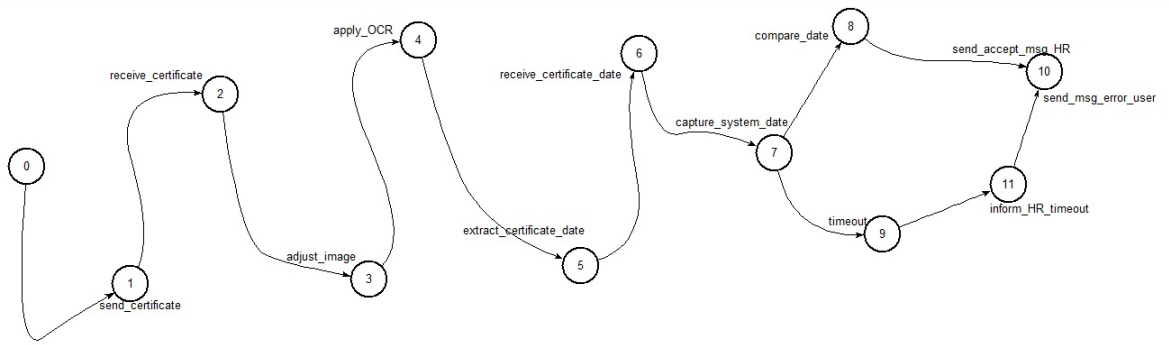


Figure 6: State Space graph of SGED Petri Nets model

to place "receiving_certificate" , which will enable the transition "adjust_image". This transition has an associated time interval of "[0, 10]", that is, the system will adjust the document within 10 seconds. After that, the transition is fired and will move the token to place "adjusting_image" .

Place "adjusting_image" will enable transition "apply_OCR", which will perform its task in up to 30 seconds according to its time interval "[0, 30]" and will move the token to place "applying_OCR". This place will enable the transition "extract_certificate-_date", which is also an immediate fire transition. When fired, "extract-_certificate_date" will move the token to place "extracting_date" (Figure 3c), which will enable transition "receive_certificate_date". This will move the token to place "receiving_date", which will enable "capture_system_date", also firing immediately.

Transition "capture_system_date" will move the token to place "capturing_date" and from there, two different situations may occur: the first situation occurs if the certificate sent is within 24 hours, the transition "compare_date" will be enabled to be fired. This transition has a time interval defined as "[0, 86400]". This time corresponds to the number of seconds within 24 hours. This transition will move the token to place "sending_msg_HR", which will enable transition "send_accept_msg_HR" and will move the token to place "save_employee_folder" .

The complete model, after merging places with same task, is shown in Figure 4.

3.2.2 Watchdog. The model shown in Figure 4 illustrates the expected operation of the SGED. However, there is a conflict situation to be foreseen. If the medical certificate is not valid, an error will occur that will be informed to the user and the certificate will be filed in his folder. This situation is caused by time constraint violation in the model.

Violation of time constraint occurs if the token remains beyond the time defined in the transition, delaying its firing. To solve conflict situations, one can use a mechanism called "watchdog" [Soares et al. 2008]. The conflict situation with the watchdog mechanism is shown in Figure 5.

If there is a violation of time restriction (medical certificate that exceeds 24 hours of issuance), the "timeout" transition is fired, firing an "alarm" and causing the exception situation. From there, an error message is sent to the HR department (token is transferred to the "alarm" location, raising awareness of the "inform_HR_timeout" transition) and a message informs the employee who sent it that the document is invalid. Firing the transition "inform_HR_timeout" takes the token to the place "informing_HR", which sensitizes transition "send_msg_error_user". After that, the document is filed in the employee's functional folder (token in place "save_employee_folder).

4 DISCUSSION

UML offers a wide variety of diagrams, but despite this, the language presents issues for visualization and simulation of system's behavior, as well as verification of properties. Therefore, Petri Nets models are developed, allowing to verify the properties of the SGED as well as verify its behavior.

By analyzing the properties, it was found that the model developed meets the properties of Reachability and Boundedness. Reachability property is discoverable in the model, for example, the place "save_employee_folder" is reachable from M_0. It is worth mentioning that place "save_employee_folder" is where the final system activity is performed. The model is considered $k - bounded$, that is, it has a finite k number of tokens in its places, which proves the Boundedness property. According to Murata's definition [Murata 1989], a network is said to be safe if it is $1 - bounded$. Therefore, the presented model can be considered a safe net, as it has only one token.

To be considered a live model, without imposing the location reached by firing the initial M_0 marking, it should be possible to fire any transition, allowing any sequence of firings. In the model shown in Figure 4, there is a single sequence of transitions firing, not meeting the Liveness property.

A Petri net model is said to be reversible if, from a M_n mark, it is possible to reach the initial mark M_0, that is, the model is able to return to its initial state. In the model presented in Figure 4, it is not possible to return to M_0. Therefore, the model is not Reversible, which is an expected property, as the Petri net model describes a workflow of business processes.

The verification of properties is relevant for the modeled system, especially those that need time constraints. Ensuring that the system meets all its purposes and respects the restrictions imposed

is essential for the correct provision of services and execution of business processes.

5 CONCLUSION

Modeling using Petri Nets with time allows one to observe various aspects of the system to be designed, verify its behavior, detect and prevent deadlocks. In this article, the bottom-up approach was used to build the model, in order to facilitate the visualization of a system named SGED regarding its components and actions.

The integration of the smaller models to form a larger one used the technique of merging places with similar tasks, creating an overview of the system as a whole. It was also possible to simulate the behavior of the system, from expected execution conditions to unwanted conditions. The verification of properties allows one to monitor what causes the system to carry out adverse conditions for its operation and carry out the necessary repairs to minimize the impacts of these conditions.

This paper shows that the use of formal methods, especially Petri Nets for modeling in practice, is quite viable, since many systems need time restrictions for certain crucial actions for their correct functioning. Large models are also possible to be developed, using a bottom-up approach that has a good correspondence with the microservices architecture, which has been widely used. It is also possible to detect and resolve situations where there are conflicting actions, creating alerts and using the watchdog technique, as shown in Figure 5, which corresponds to a violation of time constraint in the business process.

REFERENCES

Amir Abdellatif, Patrice Bonhomme, Achraf Jabeur Telmoudi, and Lotfi Nabli. 2019. Towards a Minimum Initial Marking Estimation Procedure for P-Time Labelled Petri Net Systems. In *2019 6th International Conference on Control, Decision and Information Technologies (CoDIT)*. IEEE, 42–47.

Fahad Alhumaidan and Nazir Ahmad Zafar. 2014. Possible Improvements in UML Behavior Diagrams. In *2014 International Conference on Computational Science and Computational Intelligence*, Vol. 2. 173–178. https://doi.org/10.1109/CSCI.2014.113

Tamara Almarabeh, Yousef Kh Majdalawi, and Hiba Mohammad. 2016. Cloud Computing of E-Government. (2016).

Sascha Alpers, Christoph Becker, Andreas Oberweis, and Thomas Schuster. 2015. Microservice based tool support for business process modelling. In *2015 IEEE 19th International Enterprise Distributed Object Computing Workshop*. IEEE, 71–78.

Haitham Alshibly, Raymond Chiong, and Yukun Bao. 2016. Investigating the Critical Success Factors for Implementing Electronic Document Management Systems in Governments: Evidence From Jordan. *Information Systems Management* 33, 4 (2016), 287–301.

Carlos Arévalo, Maria José Escalona, Isabel Ramos, and M Domínguez-Muñoz. 2016. A Metamodel to Integrate Business Processes Time Perspective in BPMN 2.0. *Information and Software Technology* 77 (2016), 17–33.

Mohammad Baghbani. 2019. IDEF0 Modeling Standard: a tool for process map drawing under requirements of ISO 9001: 2015: a case study. *Journal of Modern Processes in Manufacturing and Production* 8, 4 (2019), 57–66.

Armin Balalaie, Abbas Heydarnoori, Pooyan Jamshidi, Damian A Tamburri, and Theo Lynn. 2018. Microservices Migration Patterns. *Software: Practice and Experience* 48, 11 (2018), 2019–2042.

Taffarel Brant-Ribeiro, Rafael Dias Araujo, Igor Mendonça, Michel S. Soares, and Renan G. Cattelan. 2019. Interactive Web Interfaces Modeling, Simulation and Analysis using Colored Petri Nets. *Softw. Syst. Model.* 18, 1 (2019), 721–737.

Francesco Di Cerbo, Gabriella Dodero, Gianna Reggio, Filippo Ricca, and Giuseppe Scanniello. 2011. Assessing the Effectiveness of "Precise" Activity Diagrams in the Context of Business Process Modeling. In *Sistemi Evoluti per Basi di Dati - SEBD 2011, Proceedings of the Nineteenth Italian Symposium on Advanced Database Systems, Maratea, Italy, June 26-29, 2011*, Giansalvatore Mecca and Sergio Greco (Eds.). 373–380.

K.S. Cheung, T.Y. Cheung, and K.O. Chow. 2006. A Petri-Net-Based Synthesis Methodology for Use-Case-Driven System Design. *Journal of Systems and Software,* 79, 6 (2006), 772 – 790.

Søren Christensen and Laure Petrucci. 2000. Modular Analysis of Petri Nets. *The Computer Journal,* 43, 3 (2000), 224–242.

Amit Das, Harminder Singh, and Damien Joseph. 2017. A Longitudinal Study of E-Government Maturity. *Information & Management* 54, 4 (2017), 415–426.

Jennifer A. Davis, Matthew Clark, Darren Cofer, Aaron Fifarek, Jacob Hinchman, Jonathan Hoffman, Brian Hulbert, Steven P. Miller, and Lucas Wagner. 2013. Study on the Barriers to the Industrial Adoption of Formal Methods. In *Formal Methods for Industrial Critical Systems*, Charles Pecheur and Michael Dierkes (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 63–77.

Ana M Fernández-Sáez, Michel RV Chaudron, and Marcela Genero. 2013. Exploring Costs and Benefits of Using UML on Maintenance: Preliminary Findings of a Case Study in a Large IT Department. In *EESSMOD@ MoDELS*. Citeseer, 33–42.

Lidia Fuentes-Fernández and Antonio Vallecillo-Moreno. 2004. An introduction to UML profiles. *UML and Model Engineering* 2, 6-13 (2004), 72.

Claude Girault and Rudiger Valk. 2002. *Petri Nets for System Engineering: A Guide to Modeling, Verification, and Applications.* Springer-Verlag New York, Inc., Secaucus, NJ, USA.

Hejiao Huang and Helene Kirchner. 2009. Policy Composition Based on Petri Nets. In *Proceedings of the Computer Software and Applications Conference*, Vol. 2.

Pooyan Jamshidi, Claus Pahl, Nabor C Mendonça, James Lewis, and Stefan Tilkov. 2018a. Microservices: The Journey so Far and Challenges Ahead. *IEEE Software* 35, 3 (2018), 24–35.

Pooyan Jamshidi, Claus Pahl, Nabor C. Mendonça, James Lewis, and Stefan Tilkov. 2018b. Microservices: The Journey So Far and Challenges Ahead. *IEEE Software* 35, 3 (2018), 24–35. https://doi.org/10.1109/MS.2018.2141039

Tomasz Janowski. 2015. Digital Government Evolution: From Transformation to Contextualization. *Government Information Quarterly* 32, 3 (2015), 221–236.

Mu Der Jeng and F. DiCesare. 1995. Synthesis Using Resource Control Nets for Modeling Shared-Resource Systems. *IEEE Transactions on Robotics and Automation* 11, 3 (1995), 317–327.

Neil D. Jones, Lawrence H. Landweber, and Y. Edmund Lien. 1977. Complexity of Some Problems in Petri Nets. *Theoretical Computer Science* 4, 3 (1977), 277–299.

Agnes Koschmider, Andreas Oberweis, and Wolffried Stucky. 2018. A Petri net-based View on the Business Process Life-Cycle. *Enterp. Model. Inf. Syst. Archit. Int. J. Concept. Model.* 13, Special (2018), 47–55.

Alexandr Krylovskiy, Marco Jahn, and Edoardo Patti. 2015. Designing a Smart City Internet of Things Platform With Microservice Architecture. In *2015 3rd International Conference on Future Internet of Things and Cloud*. IEEE, 25–30.

Murathan Kurfalı, Ali Arifoğlu, Gül Tokdemir, and Yudum Paçin. 2017. Adoption of E-Government Services in Turkey. *Computers in Human Behavior* 66 (2017), 168–178.

S. Kurkovsky and R. Loganantharaj. 2000. Extension of Petri nets and its Applications to Model Systems with Imprecise Task Duration. In *Ninth IEEE International Conference on Fuzzy Systems. FUZZ- IEEE 2000 (Cat. No.00CH37063)*, Vol. 1. 369–374. https://doi.org/10.1109/FUZZY.2000.838688

Faming Lu, Qingtian Zeng, MengChu Zhou, Yunxia Bao, and Hua Duan. 2017. Complex Reachability Trees and their Application to Deadlock Detection for Unbounded Petri Nets. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 49, 6 (2017), 1164–1174.

Monika Malinova, Steven Gross, and Jan Mendling. 2022. A Study Into the Contingencies of Process Improvement Methods. *Information Systems* 104 (2022), 101880. https://doi.org/10.1016/j.is.2021.101880

Genc Mazlami, Jürgen Cito, and Philipp Leitner. 2017. Extraction of Microservices from Monolithic Software Architectures. In *2017 IEEE International Conference on Web Services (ICWS)*. IEEE, 524–531.

P. Merlin and D. Farber. 1976. Recoverability of Communication Protocols - Implications of a Theoretical Study. *IEEE Transactions on Communications* 24, 9 (1976), 1036–1043. https://doi.org/10.1109/TCOM.1976.1093424

Abdul Karim Mohamad, Mailasan Jayakrishnan, and Mokhtar Mohd Yusof. 2022. Thriving Information System Through Business Intelligence Knowledge Management Excellence Framework. *International Journal of Electrical & Computer Engineering (2088-8708)* 12, 1 (2022).

Tadao Murata. 1989. Petri nets: Properties, Analysis and Applications. *Proc. IEEE* 77, 4 (1989), 541–580.

Umair Mutarraf, Kamel Barkaoui, Zhiwu Li, Naiqi Wu, and Ting Qu. 2018. Transformation of Business Process Model and Notation Models onto Petri nets and their Analysis. *Advances in Mechanical Engineering* 10, 12 (2018), 1687814018808170.

Wolfgang Polak. 2002. Formal Methods in Practice. *Science of Computer Programming* 42, 1 (2002), 75–85.

Mohamed Ramadan, Hicham G Elmongui, and Riham Hassan. 2011. BPMN Formalisation Using Coloured Petri Nets. In *Proceedings of the 2nd GSTF annual international conference on software engineering & applications (SEA 2011)*. 83–90.

C. Ramchandani. 1974. *Analysis of Asynchronous Concurrent Systems by Timed Petri Nets.* Technical Report. USA.

Andreas Rogge-Solti and Mathias Weske. 2015. Prediction of Business Process Durations Using Non-Markovian Stochastic Petri Nets. *Information Systems* 54 (2015), 1–14.

Bernhard Schäfer, Margret Keuper, and Heiner Stuckenschmidt. 2021. Arrow R-CNN for handwritten diagram recognition. *International Journal on Document Analysis and Recognition (IJDAR)* 24, 1 (2021), 3–17.

Michel S. Soares and Daniel S. Cioquetta. 2012. Analysis of Techniques for Documenting User Requirements. In *Computational Science and Its Applications - ICCSA 2012 - 12th International Conference, Salvador de Bahia, Brazil, June 18-21, 2012, Proceedings, Part IV (Lecture Notes in Computer Science, Vol. 7336)*, Beniamino Murgante, Osvaldo Gervasi, Sanjay Misra, Nadia Nedjah, Ana Maria A. C. Rocha, David Taniar, and Bernady O. Apduhan (Eds.). Springer, 16–28.

Michel S. Soares, Stéphane Julia, and Jos L. M. Vrancken. 2008. Real-Time Scheduling of Batch Systems using Petri nets and Linear Logic. *Journal of Systems and Software* 81, 11 (2008), 1983–1996.

Michel S. Soares and Jos Vrancken. 2008. A Metamodeling Approach to Transform UML 2.0 Sequence Diagrams to Petri Nets. In *Proceedings of the IASTED International Conference on Software Engineering* (Innsbruck, Austria) *(SE '08)*. ACTA Press, USA, 159––164.

Michel S. Soares and Jos Vrancken. 2012. A Modular Petri Net to Modeling and Scenario Analysis of a Network of Road Traffic Signals. *Control Engineering Practice* 20, 11 (2012), 1183–1194. Special Section: Wiener-Hammerstein System Identification Benchmark.

Michel S. Soares and Jos L. M. Vrancken. 2007. Road Traffic Signals Modeling and Analysis with Petri nets and Linear Logic. In *Proceedings of the IEEE International Conference on Networking, Sensing and Control, ICNSC 2007, London, UK, 15-17 April 2007*. IEEE, 169–174.

Amrik S. Sohal and Paul Fitzpatrick. 2002. IT governance and management in large Australian organisations. *International Journal of Production Economics* 75, 1 (2002), 97–112. Information Technology/Information Systems in 21st Century Production.

Richard Swedberg. 2020. Exploratory research. *The Production of Knowledge: Enhancing Progress in Social Science* (2020), 17–41.

Pedro Valderas, Victoria Torres, and Estefanía Serral. 2022. Modelling and Executing IoT-enhanced Business Processes Through BPMN and Microservices. *Journal of Systems and Software* 184 (2022), 111139. https://doi.org/10.1016/j.jss.2021.111139

W. Vogler. 1992. *Modular Construction and Partial Order Semantics of Petri Nets*. Springer-Verlag, Secaucus, NJ, USA.

Wei Wang, Tianwa Chen, Marta Indulska, Shazia Sadiq, and Barbara Weber. 2022. Business Process and Rule Integration Approaches — An Empirical Analysis of Model Understanding. *Information Systems* 104 (2022), 101901.

Armin Zimmermann. 2012. Modeling and Evaluation of Stochastic Petri nets with TimeNET 4.1. In *6th International ICST Conference on Performance Evaluation Methodologies and Tools*. IEEE, 54–63.